

# Warum muss ich beides prüfen?

```
if (numbers == null || numbers.isEmpty()) { ... }
```

null und "leer" sind zwei völlig verschiedene Zustände sind, die unterschiedliche Probleme verursachen.

# numbers == null

- **Was es prüft:** Diese Prüfung schaut, ob die Variable numbers überhaupt auf ein Objekt verweist.
- **Was ist null?** null bedeutet "nichts". Die Variable existiert, aber sie zeigt nicht auf eine Kiste (Collection) im Speicher. Sie zeigt ins Leere.
- **Warum ist das ein Problem?** Wenn numbers null ist und du versuchst, eine Methode darauf aufzurufen (wie .isEmpty() oder .size()), stürzt dein Programm sofort mit einer NullPointerException ab.
- **Analogie:** Du fragst, ob du überhaupt eine Kiste bekommen hast.

# numbers.isEmpty()

- **Was es prüft:** Diese Prüfung schaut, ob die Collection, auf die numbers verweist, irgendwelche Elemente enthält.
- **Was ist "leer"?** "Leer" bedeutet, dass du ein *echtes, valides Collection-Objekt* hast (es ist nicht null), aber es sind einfach null Elemente darin. Die Größe ist 0.
- **Warum ist das ein Problem?** Für *diese spezifische Methode* (calculateAverageOptional) ist eine leere Liste ein Problem. Du kannst keinen Durchschnitt aus null Zahlen berechnen (das würde zu einer Division durch null führen). In diesem Fall ist es logisch korrekt, ein Optional.empty() zurückzugeben.
- **Analogie:** Du hast eine Kiste bekommen, aber sie ist leer.

# Reihenfolge

- Java wertet den Ausdruck `A || B` von links nach rechts aus:
- **Fall 1: numbers ist null**
  - Java prüft `numbers == null`.
  - Das Ergebnis ist `true`.
  - Wegen des `||` (ODER) reicht ein `true` aus, damit die gesamte Bedingung wahr ist. Java **hört sofort auf zu prüfen** und führt den Code im `if`-Block aus.
  - **Wichtig:** `numbers.isEmpty()` wird **niemals aufgerufen**. Dadurch wird die `NullPointerException` verhindert!
- **Fall 2: numbers ist eine leere Liste (nicht null, aber leer)**
  - Java prüft `numbers == null`.
  - Das Ergebnis ist `false`.
  - Da der erste Teil `false` war, *muss* Java jetzt den zweiten Teil prüfen: `numbers.isEmpty()`.
  - Dieser Aufruf ist **sicher**, da wir ja wissen, dass `numbers` nicht `null` ist.
  - `numbers.isEmpty()` ergibt `true`.
  - Die Gesamtbedingung (`false || true`) ist `true`, und der `if`-Block wird ausgeführt (was korrekt ist, da wir keinen Durchschnitt berechnen können).
- **Fall 3: numbers ist eine gefüllte Liste**
  - Java prüft `numbers == null`. Das ist `false`.
  - Java prüft `numbers.isEmpty()`. Das ist auch `false`.
  - Die Gesamtbedingung (`false || false`) ist `false`. Der `if`-Block wird übersprungen, und deine Methode kann den Durchschnitt berechnen

# Zusammenfassung:

- Du prüfst auf **null**, um einen Programmabsturz (NullPointerException) zu verhindern.
- Du prüfst auf **isEmpty()**, um einen logischen Fehler (wie Division durch null) zu verhindern und den Anwendungsfall "keine Daten" korrekt zu behandeln (indem du z.B. Optional.empty() zurückgibst).